

AES-128 Crypto Accelerator Architecture Design

Hardware Architecture & Preliminary Specification

Date	2026.04.12
Author	Ha JH
Target	Digilent Basys 3
Language	Verilog/SystemVerilog

Table of Contents (목차)

1. Introduction (개요)

2. Design Assumptions & Constraints (설계 가정 및 제약)

3. Security Considerations (보안 고려사항)

4. Performance & Resource Target (성능 및 자원 목표)

5. Overall System Architecture (전체 시스템 구조)

6. AES-128 Core Design (AES-128 코어 설계)

7. Key Expansion Design (키 확장 설계)

8. CTR Mode Engine (CTR 모드 엔진)

9. GCM/GHASH Engine (GCM/GHASH 엔진)

10. Interfaces & Timing (인터페이스 및 타이밍)

11. FSM Design (상태 머신 설계)

12. Verification & Test Plan (검증 및 테스트 계획)

13. Implementation Plan (구현 순서)

1. Introduction (개요)

1.1 Purpose (목적)

본 문서는 FPGA 기반 AES-128 GCM 암호화 가속기(Crypto Accelerator)의 초기 아키텍처 및 설계 방향을 정의한다. 본 프로젝트의 주된 목적은 AES-128 블록 암호 코어를 Verilog/SystemVerilog로 직접 구현하고, 이를 기반으로 기밀성(Confidentiality)과 무결성(Integrity)을 동시에 제공하는 고성능 AES-128 GCM 엔진의 모듈 구조를 설계하는 데 있다.

1.2 Design Goals (설계 목표)

- 초기 구현은 검증과 디버깅이 용이한 반복형(Iterative) FSM 구조로 설계하고, 기본 동작 검증 이후 처리량(Throughput) 향상을 위해 병렬화 및 파이프라인(Pipeline) 구조로 확장한다.
- CTR 모드의 특성을 활용하여 암호화와 복호화 과정 모두에 단일 AES-128 Encryption Core를 재사용하고, 이를 통해 하드웨어 면적(Area)을 최소화한다.
- AES 암호화 블록, CTR 모드 래퍼(Wrapper), GHASH 인증 블록을 독립적으로 모듈화하여 유지보수와 기능 확장이 쉬운 구조로 설계한다.

1.3 Scope (범위)

본 설계 범위는 AES-128 Encryption Core, Key Expansion 로직, CTR 모드 엔진, GHASH 연산기, Top-level Control FSM, 기본 입출력 인터페이스의 구현과 검증을 포함한다. 향후 SoC 통합을 위한 AXI4-Lite, APB 등의 Memory-mapped Bus Interface 구조는 본 코어 설계 검증이 완료된 후 후속 확장 항목으로 다룬다. ECB나 CBC 등 다른 암호화 모드 지원을 위한 복호화 전용 AES 코어 설계는 본 프로젝트의 기본 범위에서 제외한다.

1.4 Target Platform (대상 플랫폼)

Target Board	Digilent Basys 3 FPGA Board
Target Device	AMD Xilinx Artix-7
Hardware Description Language (HDL)	SystemVerilog
Development Tool	AMD Xilinx Vivado Design Suite
Target Clock Frequency	100 MHz (Basys 3 On-board Oscillator 기준)

2. Design Assumptions & Constraints (설계 가정 및 제약)

본 장에서는 초기 RTL 구현 및 검증 범위를 명확히 하기 위한 설계 가정과 제약 조건을 정의한다. 아래 기준들은 1차 설계의 복잡도를 최소화하여 핵심 알고리즘(AES, GHASH)의 동작을 확실하게 검증하기 위함이며, 향후 성능 최적화 및 버스 인터페이스 통합 단계에서 점진적으로 확장할 계획이다.

Item	Initial Assumption	Future Extension
Key Size	AES-128 (128-bit) 단일 키 크기 기준	round key 저장 및 재사용 구조 최적화
Block Size	128-bit block 단위 데이터 처리	streaming interface에서 block packing 처리
IV/Nonce	GCM 표준 권장 구조에 따라 96-bit IV 사용	non-96-bit IV의 GHASH 기반 J0 생성 지원
Input Alignment	AAD와 data는 128-bit aligned block으로 가정	partial block padding 및 byte enable 처리
Interface	block-level start/done handshake 사용	APB 또는 AXI4-Lite control interface 추가
Decrypt Path	CTR/GCM 복호화는 AES encryption core 기반 keystream XOR 구조 사용	N/A (CTR/GCM 특성상 복호화 전용 inverse datapath 불필요)

3. Security Considerations (보안 고려사항)

AES-GCM 계열 설계는 암호 알고리즘 자체뿐 아니라 Key, IV(Nonce), Tag 처리 정책이 중요하다. 본 1차 RTL 구현 단계에서는 다음 항목을 설계 제약으로 반영한다.

- 동일한 key와 동일한 IV/nonce/counter 조합은 재사용하지 않는다.
- GCM 복호화에서 Tag Mismatch가 발생하면 plaintext 출력을 무효화하고 보안 에러 status flag를 발생시킨다.
- reset 또는 key update 시 key register와 round key register를 clear하도록 설계한다.
- debug 또는 status 신호를 통해 secret key, round key, 내부 GHASH 중간값이 외부로 노출되지 않도록 한다.
- 초기 버전에는 side-channel countermeasure를 포함하지 않으며, masking 또는 hiding 기법은 향후 보안 강화 항목으로 둔다.

GCM에서 IV/nonce 재사용은 보안성을 크게 손상시킬 수 있다. 따라서 시스템 수준에서는 nonce 생성과 재사용 방지 정책을 암호화 IP 외부의 firmware 또는 host controller와 함께 정의해야 한다.

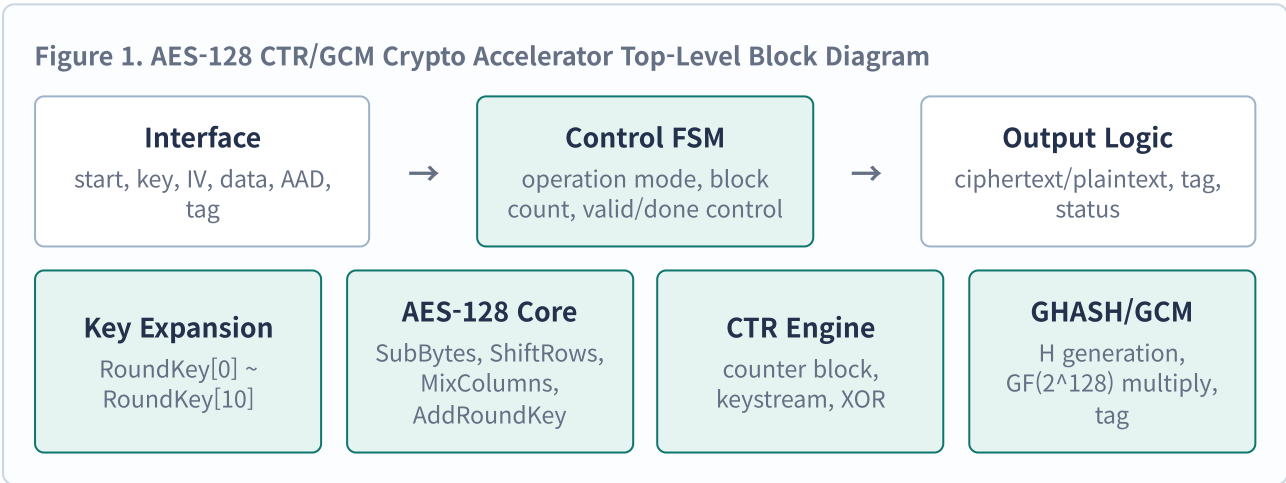
4. Performance & Resource Target (성능 및 자원 목표)

초기 구현은 최고 성능 구현보다 아키텍처 구조 이해, 기능 검증, 확장 가능성 확보를 우선한다. 1차 버전은 iterative AES core를 기준으로 설계하고, 검증 완료 이후 S-box 병렬화, round unroll, pipeline 구조를 단계적으로 확장한다.

Item	Initial Target	Optimization Direction
AES Latency	약 10~12 cycles/block 목표	round pipeline 또는 partial unroll 적용 계획
S-box Structure	16-byte 병렬 lookup table 우선 고려	resource 절약형 단일 S-box 재사용 구조 비교
Key Expansion	precompute 후 round key 재사용	key change 빈도에 따른 key cache 구조 검토
CTR Throughput	1 AES block 완료마다 128-bit data 처리	pipeline AES 구조로 처리량 향상
GHASH	iterative GF(2 ¹²⁸) multiplier로 시작	Karatsuba 또는 parallel multiplier 검토

5. Overall System Architecture (전체 시스템 구조)

최상위 가속기 모듈은 제어 로직(Control FSM), 데이터 패스(AES-128 Core, Key Expansion, CTR Engine, GHASH/GCM Engine), 그리고 외부와의 통신을 담당하는 Interface Block으로 구성된다. 1차 설계 버전에서는 검증 편의성을 위해 128-bit 블록 단위의 단순 핸드셰이크(valid/ready) 인터페이스를 채택하고, 이후 streaming interface 또는 AXI4 계열 bus interface로 확장 가능한 아키텍처를 지향한다.



5.1 Main Modules (주요 모듈)

Module	Role	Initial Implementation
<code>aes128_core</code>	128-bit block AES encryption 수행	10-round iterative FSM
<code>aes_key_expand</code>	128-bit key에서 11개 round key 생성	combinational 또는 precompute 방식
<code>ctr_engine</code>	counter block 생성 및 keystream XOR 기반 데이터 처리	block_count 기반 순차 처리
<code>ghash_engine</code>	AAD와 ciphertext를 이용한 GHASH 인증값 계산	GF(2 ¹²⁸) iterative multiplier

Module	Role	Initial Implementation
<code>gcm_engine</code>	CTR 데이터 처리와 GHASH 기반 tag 생성/검증 통합	encrypt/decrypt mode FSM

6. AES-128 Core Design (AES-128 코어 설계)

AES-128은 128-bit block과 128-bit key를 사용하며 총 10개의 round로 구성된다. 본 설계의 초기 버전은 하나의 round datapath를 여러 클럭에 걸쳐 재사용하는 iterative 구조로 설계한다.

6.1 AES-128 Overview

AES-128은 128-bit plaintext와 128-bit secret key를 입력으로 받아 128-bit ciphertext를 생성하는 대칭키 블록 암호이다. 내부적으로 16-byte state를 사용하며, key expansion을 통해 생성한 round key를 각 round에서 state와 XOR한다. 본 설계에서는 AES-128 encryption core를 기본 연산 블록으로 사용하고, CTR/GCM 모드에서도 동일한 core를 재사용한다.

6.2 AES Encryption Flow

```

Initial Round 0:
    state = plaintext XOR round_key[0]

Round 1 ~ 9:
    state = SubBytes(state)
    state = ShiftRows(state)
    state = MixColumns(state)
    state = AddRoundKey(state, round_key[round])

Final Round 10:
    state = SubBytes(state)
    state = ShiftRows(state)
    state = AddRoundKey(state, round_key[10])
  
```

6.3 Internal Module Partition

Submodule	Input	Output	Description
<code>aes_sbox</code>	8-bit byte	8-bit byte	AES 표준 S-box lookup table
<code>aes_subbytes</code>	128-bit state	128-bit state	16개 byte에 S-box 적용
<code>aes_shiftrows</code>	128-bit state	128-bit state	AES state row 단위 byte rotation
<code>aes_mixcolumns</code>	128-bit state	128-bit state	GF(2 ⁸) 기반 column mixing

Submodule	Input	Output	Description
<code>aes_addroundkey</code>	state, round key	128-bit state	state와 round key XOR

6.4 State Byte Mapping

AES는 내부적으로 128-bit(16-byte) state를 4x4 2차원 배열로 나누어 처리한다. 구현 초기에 byte mapping을 명확히 고정해야 ShiftRows, MixColumns, test vector 검증에서 혼선을 줄일 수 있다.

본 설계는 표준 Column-Major Mapping을 따른다. 128-bit AES 입력 `state_in[127:0]` 에 대해 최상위 바이트(MSB)인 `state_in[127:120]` 을 state 배열의 `row0`, `col0` 위치로 매핑한다.

	Column 0	Column 1	Column 2	Column 3
Row 0	Byte 0 <code>[127:120]</code>	Byte 4 <code>[95:88]</code>	Byte 8 <code>[63:56]</code>	Byte 12 <code>[31:24]</code>
Row 1	Byte 1 <code>[119:112]</code>	Byte 5 <code>[87:80]</code>	Byte 9 <code>[55:48]</code>	Byte 13 <code>[23:16]</code>
Row 2	Byte 2 <code>[111:104]</code>	Byte 6 <code>[79:72]</code>	Byte 10 <code>[47:40]</code>	Byte 14 <code>[15:8]</code>
Row 3	Byte 3 <code>[103:96]</code>	Byte 7 <code>[71:64]</code>	Byte 11 <code>[39:32]</code>	Byte 15 <code>[7:0]</code>

7. Key Expansion Design (키 확장 설계)

AES-128 key expansion은 128-bit 원본 key로부터 11개의 128-bit round key를 생성한다. 초기 구현에서는 AES 연산 시작 전에 모든 round key를 생성하여 내부 register 배열에 저장하는 구조를 사용한다.

7.1 Round Key Structure

Round Key	Usage	Note
<code>round_key[0]</code>	Initial AddRoundKey	원본 key와 동일
<code>round_key[1] ~ round_key[9]</code>	Main rounds	SubBytes, ShiftRows, MixColumns 이후 XOR
<code>round_key[10]</code>	Final round	MixColumns가 생략된 최종 State와 XOR

7.2 Implementation Direction

- 초기 버전은 combinational key expansion으로 작성하여 AES core 검증을 단순화한다.

- timing path가 길어질 경우 key expansion FSM을 분리하고 round key memory 또는 register file을 사용한다.
- 동일 key로 여러 block을 처리할 경우 round key를 재사용하여 불필요한 재계산을 줄인다.

CTR/GCM에서는 같은 key로 여러 counter block을 AES 암호화하므로, key expansion 결과를 한 번 생성한 뒤 여러 block에서 재사용하는 구조가 효율적이다.

8. CTR Mode Engine (CTR 모드 엔진)

CTR 모드에서는 plaintext를 AES core에 직접 넣지 않는다. IV/nonce와 counter로 구성된 counter block을 AES encryption core로 암호화해 keystream을 만들고, 이 keystream과 입력 데이터를 XOR해 출력 데이터를 생성한다.

8.1 CTR Mode Overview

CTR(Counter) 모드는 블록 암호를 stream cipher처럼 사용할 수 있게 하는 운용 모드이다. AES core는 counter block을 암호화해 keystream을 만들고, 실제 입력 데이터는 이 keystream과 XOR되어 암호문 또는 평문으로 변환된다. 이 구조에서는 암호화와 복호화가 동일한 XOR 연산을 사용하므로, AES encryption core만으로 양방향 처리가 가능하다.

```
counter_block = nonce || counter
keystream     = AES_Encrypt(key, counter_block)
data_out      = data_in XOR keystream
```

8.2 CTR Characteristics

- 암호화와 복호화가 동일한 XOR 구조를 사용한다.
- 같은 key와 같은 nonce/counter 조합은 절대 재사용하면 안 된다.
- 여러 block 처리 시 block마다 counter를 1씩 증가시킨다.

8.3 Counter Block Format

Field	Width	Description
nonce	96-bit	message마다 고유해야 하는 IV/nonce 영역
counter	32-bit	block 단위 증가 counter
counter_block	128-bit	{nonce, counter}

9. GCM/GHASH Engine (GCM/GHASH 엔진)

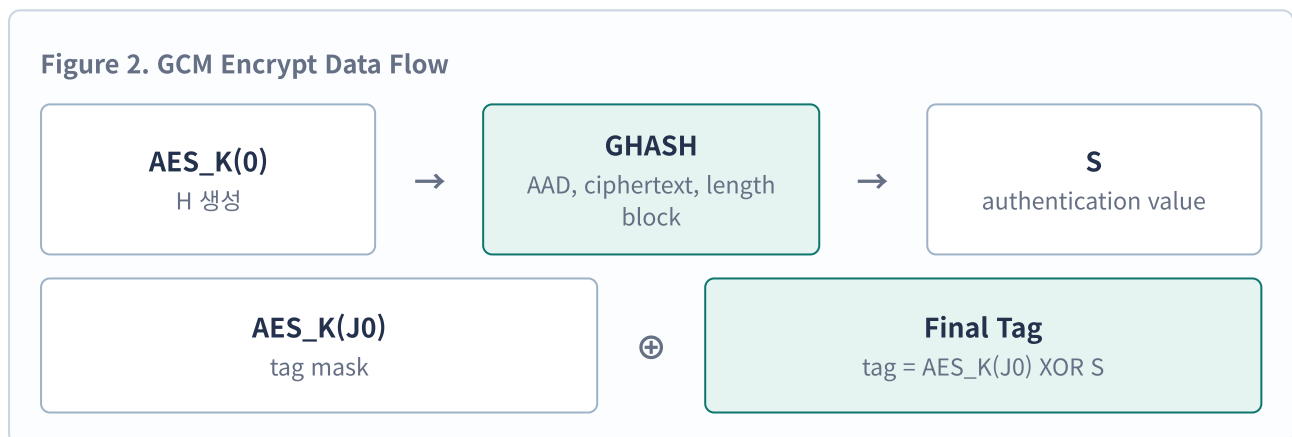
GCM(Galois/Counter Mode)은 CTR 모드의 데이터 암호화(기밀성)와 GHASH 연산을 통한 인증 태그 생성(무결성)을 결합한 인증 암호화(Authenticated Encryption) 모드이다. 가속기는 Ciphertext와 Authentication Tag를 함께 출력하며, 복호화 시에는 입력 Tag와 내부에서 재계산한 Tag를 비교하여 데이터의 변조 여부를 검증한다.

9.1 GCM/GHASH Overview

GCM 구조에서 데이터 암호화는 CTR 모드가 담당하고, 인증 태그 생성은 GHASH가 담당한다. GHASH는 AAD와 ciphertext, length block을 입력으로 받아 $GF(2^{128})$ 기반 누적 곱셈을 수행하며, 최종 결과는 `AES_K(J0)` 와 XOR되어 authentication tag가 된다.

AAD(Additional Authenticated Data)는 암호화하지 않지만 인증에는 포함되는 데이터이다. GCM 복호화에서는 ciphertext와 AAD로 tag를 다시 계산하고, 입력 tag와 비교하여 데이터가 변조되지 않았는지 확인한다.

9.2 GCM Data Flow



9.3 Important Values

Symbol	Meaning	Generation
<code>H</code>	GHASH hash subkey	<code>AES_K(128'h0)</code>
<code>J0</code>	initial counter block for tag mask	96-bit IV 사용 시 <code>IV 32'h00000001</code>
<code>CTR start</code>	encryption counter start value	96-bit IV 사용 시 <code>IV 32'h00000002</code>
<code>S</code>	GHASH output	AAD, ciphertext, length block 누적 연산

9.4 GHASH Operation

```

Y0 = 0
Yi = (Y(i-1) XOR Xi) * H over GF(2^128)
S = Yn

```

x_i 는 AAD block, ciphertext block, 마지막 length block을 순서대로 나타낸다. 마지막 length block에는 AAD bit length와 ciphertext bit length가 포함된다.

9.5 Hardware Implementation Note (하드웨어 구현 고려사항)

GHASH의 핵심 연산인 $*$ H 는 일반적인 정수 곱셈이 아니라 $GF(2^{128})$ 상의 다항식 곱셈(Polynomial Multiplication)과 축소(Reduction) 연산으로 구현된다. $GF(2^{128})$ 곱셈기는 하드웨어 자원 사용량과 critical path에 큰 영향을 줄 수 있으므로, 1클럭 조합논리(Combinational) 구조와 여러 클럭에 나누어 처리하는 순차(Sequential) 구조를 비교하여 면적과 타이밍을 함께 고려한다.

10. Interfaces & Timing (인터페이스 및 타이밍)

초기 인터페이스는 128-bit block 단위 valid/ready handshake를 사용한다. 이후 bus interface를 추가하면 이 신호들은 control/status register와 data register로 매핑할 수 있다.

10.1 Top-Level Signal Definition

Signal Name	Direction	Width	Description
clk	Input	1	system clock
rst_n	Input	1	active-low reset
start	Input	1	operation start pulse
mode	Input	2	00: AES block, 01: CTR, 10: GCM encrypt, 11: GCM decrypt
block_count	Input	16	transaction 내 128-bit data block 수
key	Input	128	AES-128 secret key
iv	Input	96	GCM/CTR nonce or IV
data_in	Input	128	plaintext, ciphertext, or AAD block
data_in_valid	Input	1	data_in block valid
data_in_ready	Output	1	engine can accept next input block

Signal Name	Direction	Width	Description
<code>data_out</code>	Output	128	ciphertext or plaintext block
<code>data_out_valid</code>	Output	1	<code>data_out</code> block valid
<code>data_out_ready</code>	Input	1	receiver can accept next output block
<code>tag</code>	Output	128	GCM authentication tag
<code>tag_valid</code>	Output	1	GCM tag output valid
<code>busy</code>	Output	1	operation in progress
<code>done</code>	Output	1	operation complete pulse
<code>tag_match</code>	Output	1	GCM decrypt tag comparison result

예를 들어 64-byte data는 `block_count = 4` 로 설정하고, `data_in_valid / data_in_ready` handshake에 따라 128-bit block 4개를 연속 입력한다. 출력 data block은 `data_out_valid / data_out_ready` handshake로 전달되며, GCM tag는 `tag_valid` 와 함께 별도 출력한다.

10.2 AES Core Port Example

```
module aes128_core (
    input logic      clk,
    input logic      rst_n,
    input logic      start,
    input logic [127:0] plaintext,
    input logic [127:0] key,
    output logic [127:0] ciphertext,
    output logic      busy,
    output logic      done
);
```

11. FSM Design (상태 머신 설계)

11.1 AES Core FSM

State	Operation	Next Condition
<code>IDLE</code>	start 대기, busy=0	<code>start == 1</code>

State	Operation	Next Condition
KEY_EXPAND	round key 생성 또는 로드	round key ready
INIT	state = plaintext XOR round_key[0]	next clock
ROUND	round 1~9 수행	round == 9
FINAL	round 10 수행, MixColumns 제외	next clock
DONE	ciphertext valid, done pulse	return to IDLE

11.2 CTR Engine FSM

```

IDLE
-> LOAD_BLOCK
-> START_AES
-> WAIT_AES_DONE
-> XOR_OUTPUT
-> INC_COUNTER
-> NEXT_BLOCK or DONE

```

11.3 GCM Engine FSM

```

IDLE
-> GEN_H
-> GEN_J0_MASK
-> PROCESS_AAD
-> CTR_CRYPT_AND_GHASH_CIPHERTEXT
-> GHASH_LENGTH_BLOCK
-> FINAL_TAG
-> TAG_COMPARE or DONE

```

데드락 대응용 watchdog은 AES/GCM 데이터패스 내부에 포함하지 않고, 각 FSM은 busy, done, error/status 신호를 제공한다. 장시간 busy 상태의 timeout 감시는 향후 CPU 또는 bus interface 수준의 상위 제어 로직에서 처리한다.

12. Verification & Test Plan (검증 및 테스트 계획)

검증은 작은 모듈부터 독립적으로 진행한 뒤 통합 모듈로 확장한다. 특히 AES state mapping과 GCM counter 초기값은 오류가 자주 발생하는 지점이므로 test vector 기반 검증을 반드시 수행한다.

12.1 AES-128 Known Answer Test

Item	Value
Key	000102030405060708090a0b0c0d0e0f
Plaintext	00112233445566778899aabbccddeeff
Expected Ciphertext	69c4e0d86a7b0430d8cdb78070b4c55a

12.2 Verification Items

- `aes_sbox` : 256-entry lookup output 확인
- `aes_shiftrows` : byte position 이동 확인
- `aes_mixcolumns` : $GF(2^8)$ multiplication 결과 확인
- `aes_key_expand` : round key sequence 확인
- `aes128_core` : AES known answer test 통과
- `ctr_engine` : encrypt 후 decrypt 시 원문 복구 확인
- `gf128_mult` : GHASH용 $GF(2^{128})$ 곱셈 검증
- `gcm_engine` : ciphertext와 tag test vector 검증

12.3 Verification Strategy

초기 검증은 Python 기반 golden reference model과 간단한 SystemVerilog testbench를 함께 사용한다. Python reference model은 AES/CTR/GCM의 기대 출력값을 생성하고, RTL testbench는 실제 RTL 출력과 기댓값을 비교한다.

Phase	Method	Description
Phase 1	Python golden model	AES, CTR, GHASH, GCM의 reference output 생성
Phase 2	Simple SystemVerilog testbench	module 단위 directed test 및 known answer test 수행
Phase 3	Directed/random test vectors	여러 key, IV, block count, AAD length 조합 검증
Phase 4	UVM-based verification	valid/ready handshake, multi-block transaction, mode switching, APB/AXI wrapper 등 full-system 검증으로 확장

UVM 환경은 초기 AES core 검증 단계의 필수 항목이 아니라, top-level interface와 bus wrapper가 추가된 이후 transaction-level 검증을 위해 확장하는 항목으로 둔다.

13. Implementation Plan (구현 순서)

Step	Target	Deliverable
1	AES basic functions	<code>aes_sbox.sv</code> , <code>aes_shiftrows.sv</code> , <code>aes_mixcolumns.sv</code>
2	Key expansion	<code>aes_key_expand.sv</code> , round key testbench
3	AES core integration	<code>aes128_core.sv</code> , AES KAT testbench
4	CTR mode	<code>ctr_engine.sv</code> , multi-block testbench
5	GHASH	<code>gf128_mult.sv</code> , <code>ghash_engine.sv</code>
6	GCM integration	<code>gcm_engine.sv</code> , tag generation/compare testbench
7	System interface	simple register interface, later APB/AXI4-Lite wrapper
8	Pipeline optimization	round pipeline or partial unroll AES core, throughput comparison

1차 milestone은 “AES-128 단일 block encryption이 NIST test vector와 일치하는 것”으로 둔다. 이 단계가 안정화된 뒤 CTR/GCM 구조를 통합하고, 이후 pipeline 최적화를 별도 milestone으로 확장한다.